МИНОБРНАУКИ РОССИИ
САНКТ-ПЕТЕРБУРГСКИЙ ГОСУДАРСТВЕННЫЙ
ЭЛЕКТРОТЕХНИЧЕСКИЙ УНИВЕРСИТЕТ
«ЛЭТИ» ИМ. В. И. УЛЬЯНОВА (ЛЕНИНА)
Кафедра МО ЭВМ

ОТЧЕТ

по лабораторной работе №1

по дисциплине «Вычислительная математика»

Тема: Решение нелинейных уравнений. Метод бисекции.

Студент гр. 4342 _____ Алюкин Г.А.

Преподаватель _____ Пуеров Г.Ю.

Санкт-Петербург

2025

Цель работы.

Исследование свойств метода бисекции при решении нелинейных уравнений. В ходе работы требуется реализовать алгоритм поиска корня, провести вычислительный эксперимент по оценке скорости сходимости (зависимость итераций от ϵ) и изучить влияние возмущений в исходных данных на точность результата. Дополнительно исследуется устойчивость процесса при вычислении заданного числа итераций в условиях моделируемых погрешностей округления функции.

Постановка задачи.

Порядок выполнения работы должен быть следующим:

1. Графически или аналитически отделить корень уравнения (т.е. найти отрезки [Left, Right], на которых функция удовлетворяет условиям теоремы Коши).

2. Составить подпрограмму вычисления функции.

3. Составить головную программу, содержащую обращение к подпрограмме f(x), BISECT, Round и индикацию результатов.

4. Провести вычисления по программе. Построить график зависимости числа итераций от Eps.

Вариант 1: f(x)=x^2 – 5.0 * sin(x)

Теоретический минимум.

Если найден отрезок [a,b], такой, что (a)(b)<0, существует точка c, в которой значение функции равно нулю, т. е. f(c)=0, c∈(a,b). Метод бисекции состоит в построении последовательности вложенных друг в друга отрезков, на концах которых функция имеет разные знаки. Каждый последующий отрезок получается делением пополам предыдущего. Процесс f(x) построения последовательности отрезков позволяет найти нуль функции (корень уравнения с любой заданной точностью.

Рассмотрим один шаг итерационного процесса. Пусть на (n-1)-м шаге найден отрезок [an-1, bn-1]⊂[a, b], такой, что f(an-1) f(bn-1)<0. Разделим его пополам точкой ξ=(an-1 +bn-1)/2 и вычислим f(x). Если f(ξ)=0, то ξ=(an-1+bn-1)/2- корень уравнения. Если f(ξ)≠0, то из двух половин отрезка выбирается та, на концах которой функция имеет противоположные знаки, поскольку искомый корень лежит на этой половине, т.е.

an=an-1, bn=ξ , если f (ξ)f(an-1) < 0 ;

an=ξ, bn= bn-1 , если f (ξ)f(an-1) > 0 .

Если требуется найти корень с точностью ε, то деление пополам продолжается до тех пор, пока длина отрезка не станет меньше 2ε. Тогда координата середины отрезка есть значение корня с требуемой точностью ε.

Метод бисекции является простым и надежным методом поиска простого корня уравнения f(x) = 0 (простым называется корень x=c дифференцируемой функции f(x), если f(c) и f'(c)≠0). Этот метод сходится для любых непрерывных функций f(x), в том числе недифференцируемых. Скорость его сходимости невысока. Для достижения точности e необходимо совершить N≈log2(b-a)/ε итераций. Это означает, что для получения каждых трех верных десятичных знаков необходимо совершить около 10 итераций.

В лабораторной работе №3 предлагается, используя программы - функции BISECT и Round из файла methods.cpp (файл заголовков metods.h, директория LIBR1), найти корень уравнения f(x)=0 методом бисекции с заданной

точностью Eps, исследовать зависимость числа итераций от точности Eps при изменении Eps от 0.1 до 0.000001, исследовать обусловленность метода (чувствительность к ошибкам в исходных данных).

Выполнение работы.

1. Отделение корня уравнения (т.е. нахождение отрезков [Left, Right], на которых функция удовлетворяет условиям теоремы Коши).

Функция:

$$f(x)=x^2 – 5\sin(x)$$

Для отделения корней воспользуемся свойством непрерывной функции: если на концах отрезка функция имеет разные знаки, то внутри отрезка существует хотя бы один корень.

Подставляем разные значения x и смотрим на знак результата:

x	Результат f(x)	Знак
-1	≈ 5.2	+
0	0	Корень
1	≈ -3.2	-
2	≈ -0.55	-
3	≈ 8.3	+

Вычисляем значения функции:

$f(2)=2^2 – 5\sin(2) \approx -0.55$ - отрицательное

$f(3)=3^2 – 5\sin(3) \approx 8.30$ - положительное

Между x = 2 (функция ниже нуля) и x = 3 (выше нуля) происходит изменение знака. Корень лежит на отрезке [2, 3], следовательно принимаем данный отрезок в качестве начального для метода бисекции.

[Left; Right] = [0; 0.5].

2. Подпрограмма вычисления функции f(x).

Для работы метода бисекции необходима подпрограмма, вычисляющая f(x) для произвольного x. Эта функция вызывается на каждой итерации для определения значений в граничных точках и середине текущего интервала.

В lab3.cpp была переопределена функция extern double F(double) в соответствии с формулой:

$$f(x)=x^2 - 5\sin(x)$$

Где:

x — вещественный аргумент

sin(x) - тригонометрическая функция синус (аргумент x в радианах);

Исходный код в Приложении А.

3. Составление головной программы, содержащей обращение к подпрограмме f(x), BISECT, Round и индикацию результатов.

Создана функция находящаяся в файле lab3.cpp в Приложении А.

4. Провести вычисления по программе. Построить график зависимости числа итераций от Eps.

На основе выполнения алгоритма метода бисекции получаем следующие результаты:

Точность (ε)	Корень (x)	Число итераций (N)
10^{-1} (0.1)	2.1562500	4
10^{-2} (0.01)	2.1132813	7
10^{-3}	2.1108398	10
10^{-4}	2.1105347	14
10^{-5}	2.1105118	17
10^{-6}	2.1105111	20

Построение графика зависимости N от Eps.

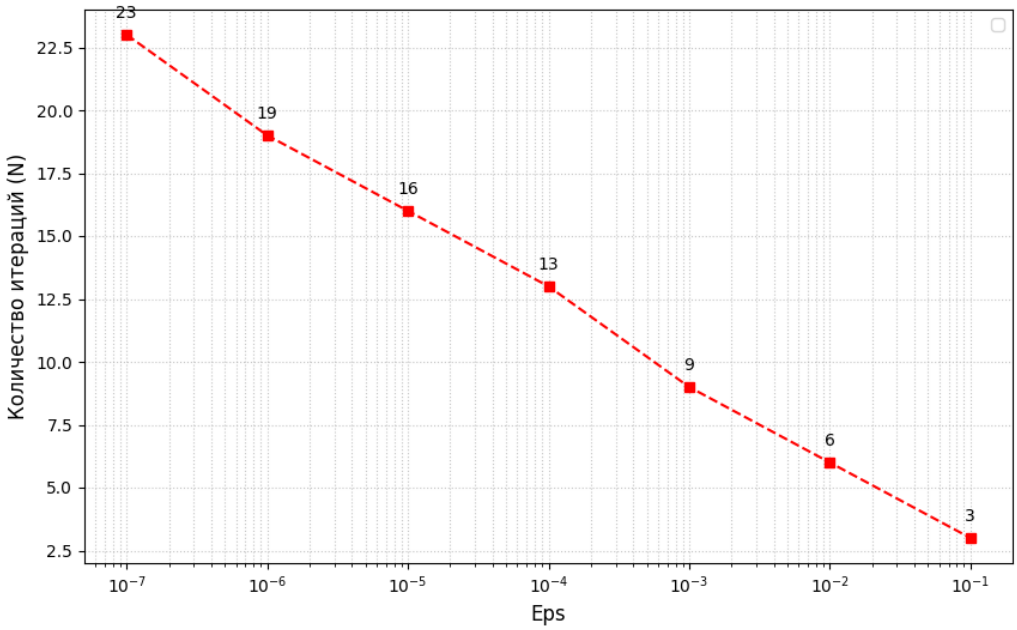

Рисунок 1 — График зависимости N от точности вычислений

5. Исследование чувствительности метода к ошибкам в исходных данных (моделирование Round)

Чтобы исследовать чувствительность метода к ошибкам в исходных данных, было проведено исследование с разными Delta и Eps.

Результаты вычислений приведены ниже.

Delta	Eps	Значение BISECT	N
10^{-1}	10^{-1}	2.125000	3
	10^{-2}	2.078125	6
	10^{-3}	2.085938	6
	10^{-4}	2.085938	6
	10^{-5}	2.085938	6
	10^{-6}	2.085938	6
10^{-3}	10^{-1}	2.125000	3
	10^{-2}	2.078125	6
	10^{-3}	2.085938	6
	10^{-4}	2.085938	6
	10^{-5}	2.085938	6
	10^{-6}	2.085938	6

10^{-5}	10^{-1}	2.125000	3
	10^{-2}	2.078125	6
	10^{-3}	2.083984	9
	10^{-4}	2.085815	13
	10^{-5}	2.085922	16
	10^{-6}	2.085936	19
10^{-7}	10^{-1}	2.125000	3
	10^{-2}	2.078125	6
	10^{-3}	2.083984	9
	10^{-4}	2.085815	13
	10^{-5}	2.085922	16
	10^{-6}	2.085936	19

Исследование показало, что достижимая точность поиска корня напрямую ограничена погрешностью вычисления функции (Delta). Если требовать точность Eps выше, чем значение Delta, уточнение корня прекращается, а число итераций перестает расти.

Таким образом применение функции Round позволяет исследовать зависимость точности решения от ошибок округления. Сравнительный анализ результатов при изменении шага Delta дает возможность определить чувствительность алгоритма к точности представления чисел и выявить границы метода.

Вывод.

В ходе выполнения лабораторной работы был изучен и программно реализован метод бисекции для нахождения корня нелинейного уравнения f(x)=x^2 – 5.0 * sin(x) на отрезке [2, 3]. Подтверждена линейная скорость сходимости метода: график зависимости числа итераций N от точности Eps является прямой линией. Также в ходе исследования было установлено, что на точность поиска корня влияет погрешность вычисления самой функции (Delta). Таким образом, метод бисекции является крайне надежным и устойчивым алгоритмом, однако его точность всегда ограничена качеством (точностью) задания исходных данных.

ПРИЛОЖЕНИЕ А
ИСХОДНЫЙ КОД ПРОГРАММЫ

Файл lb3.cpp:

```
#include <stdio.h>
#include <math.h>
#include <stdlib.h>
#include "methods.h"

double delta;

double F(double x)
{
    extern double delta;
    double s;

    s = x * x - 5.0 * sin(x); // Значение функции

    s = Round(s, delta); // Округление с точностью delta

    return s;
}

int main(){
    double a, b, eps, x;
    int k;
    float a1, b1, eps1, delta1;

    printf("Лабораторная работа №3. Вариант 1\n");
    printf("Функция: f(x) = x^2 - 5*sin(x)\n\n");

    a = 2;
    b = 3;
```

```
    eps = 0.000001; //Точность вычисления корня
    delta = 0.0000001; // Точность задания функции

    x = BISECT(a, b, eps, k);

    printf("\nРезультаты:\n");
    printf("Корень x = %f\n", x);
    printf("Число итераций N = %d\n", k);
    return 0;
}
```

Файл methods.cpp:

```
#include <stdio.h>
#include <math.h>
#include <stdlib.h>

extern double F(double);
/*
МЕТОД БИСЕКЦИИ (Половинного деления)
Left, Right - границы интервала
Eps - точность
N - переменная для возврата количества итераций
*/
double BISECT(double Left, double Right, double Eps, int &N)
{
    double E = fabs(Eps) * 2.0;
    double FLeft = F(Left);
    double FRight = F(Right);
    double X = (Left + Right) / 2.0;
    double Y;

    if (FLeft * FRight > 0.0) {
        puts("Неверное задание интервала\n");
        exit(1);
```

```
        }
        if (Eps <= 0.0) {
            puts("Неверное задание точности\n");
            exit(1);
        }

        N = 0;
        if (FLeft == 0.0)  return Left;
        if (FRight == 0.0) return Right;

        while ((Right - Left) >= E)
        {
               X = 0.5 * (Right + Left); // Вычисление середины
отрезка
            Y = F(X);

            if (Y == 0.0) return (X);

            if (Y * FLeft < 0.0) {
                Right = X;
            } else {
                Left = X;
                FLeft = Y;
            }
            N++;
        }
        return (X);
    }

    /*
    ФУНКЦИЯ ОКРУГЛЕНИЯ
    X - число
    Delta - шаг округления (например, 0.01)
    */
    double Round(double X, double Delta)
```

```
{
    if (Delta <= 1E-9) {
        puts("Неверное задание точности округления\n");
        exit(1);
    }
    if (X > 0.0)
        return (Delta * (long((X / Delta) + 0.5)));
    else
        return (Delta * (long((X / Delta) - 0.5)));
}
```

Файл methods.h

```
#include "methods.cpp"

extern double F(double);
double Round (double X,double Delta);
double BISECT(double Left, double Right, double Eps, int &N);
```